

Kanbien packages/core Architecture Guide

Shared Core Foundations, Enterprise Modules, Async Systems, Tenancy,
Localization, and v1 Recommendations

22 June 2026

Contents

Part 1: Foundations	7
What a shared core package is	7
Why shared core packages exist	7
What problems shared core packages solve	7
What problems shared core packages create when used incorrectly	8
How core differs from applications, features, platform, infrastructure, design systems, and utilities	8
Why enterprise teams create a shared core layer	8
When a shared core layer becomes a dumping ground and how to prevent it	9
Part 2: Understanding the Kanbien Architecture	10
Conceptual stack	10
Code dependency direction	11
What depends on what	11
What should never depend on what	11
Why dependency direction matters	12
How poor dependency management causes architectural decay	12
Part 3: Designing packages/core	13
async-jobs/	13
authn/	14
authz/	15
notifications/	16
validation/	17
persistence/	17
files/	18
events/	19
queues/	20
logging/	21
monitoring/	22
audit/	23
reporting/	24
analytics/	25
config/	26
tenancy/	27
security/	28
i18n/	28
localization/	29
Part 4: Deep Dive into Every Module	31
async-jobs/ deep dive	31
Why it exists	31
How it works internally	31
Example implementation	31
Example Kanbien usage	31
Enterprise concerns	32
Architectural warning	32
authn/ deep dive	32
Why it exists	32
How it works internally	32
Example implementation	32
Example Kanbien usage	33
Enterprise concerns	33
Architectural warning	33
authz/ deep dive	33

Why it exists	33
How it works internally	33
Example implementation	33
Example Kanbien usage	34
Enterprise concerns	34
Architectural warning	34
notifications/ deep dive	34
Why it exists	34
How it works internally	34
Example implementation	35
Example Kanbien usage	35
Enterprise concerns	35
Architectural warning	35
validation/ deep dive	36
Why it exists	36
How it works internally	36
Example implementation	36
Example Kanbien usage	36
Enterprise concerns	36
Architectural warning	36
persistence/ deep dive	37
Why it exists	37
How it works internally	37
Example implementation	37
Example Kanbien usage	37
Enterprise concerns	37
Architectural warning	38
files/ deep dive	38
Why it exists	38
How it works internally	38
Example implementation	38
Example Kanbien usage	38
Enterprise concerns	39
Architectural warning	39
events/ deep dive	39
Why it exists	39
How it works internally	39
Example implementation	39
Example Kanbien usage	39
Enterprise concerns	40
Architectural warning	40
queues/ deep dive	40
Why it exists	40
How it works internally	40
Example implementation	40
Example Kanbien usage	41
Enterprise concerns	41
Architectural warning	41
logging/ deep dive	41
Why it exists	41
How it works internally	41
Example implementation	41
Example Kanbien usage	42
Enterprise concerns	42
Architectural warning	42
monitoring/ deep dive	42
Why it exists	42

How it works internally	42
Example implementation	42
Example Kanbien usage	42
Enterprise concerns	43
Architectural warning	43
audit/ deep dive	43
Why it exists	43
How it works internally	43
Example implementation	43
Example Kanbien usage	44
Enterprise concerns	44
Architectural warning	44
reporting/ deep dive	44
Why it exists	44
How it works internally	44
Example implementation	44
Example Kanbien usage	45
Enterprise concerns	45
Architectural warning	45
analytics/ deep dive	45
Why it exists	45
How it works internally	45
Example implementation	46
Example Kanbien usage	46
Enterprise concerns	46
Architectural warning	46
config/ deep dive	46
Why it exists	46
How it works internally	47
Example implementation	47
Example Kanbien usage	47
Enterprise concerns	47
Architectural warning	47
tenancy/ deep dive	47
Why it exists	47
How it works internally	47
Example implementation	48
Example Kanbien usage	48
Enterprise concerns	48
Architectural warning	48
security/ deep dive	48
Why it exists	48
How it works internally	49
Example implementation	49
Example Kanbien usage	49
Enterprise concerns	49
Architectural warning	49
i18n/ deep dive	50
Why it exists	50
How it works internally	50
Example implementation	50
Example Kanbien usage	50
Enterprise concerns	50
Architectural warning	51
localization/ deep dive	51
Why it exists	51
How it works internally	51

Example implementation	51
Example Kanbién usage	51
Enterprise concerns	51
Architectural warning	52
Part 5: Events, Messaging, and Async Systems	53
Event-driven architecture	53
Domain events	53
Event buses	53
Message queues	53
Dead-letter queues	53
Background workers	53
Async jobs	54
Scheduled jobs	54
How they relate	54
What belongs in core, apps, platform, and infra	54
Complete worked example: UserCreated	55
1. API receives request	55
2. App creates user and records an outbox event	56
3. Event publisher publishes UserCreated	56
4. Event subscriber creates a queue job	56
5. Worker processes the job	57
6. Audit is recorded	57
7. Analytics is tracked	57
Full flow diagram	58
Part 6: Reporting vs Analytics vs Audit	59
Reporting	59
Analytics	59
Business Intelligence	59
Audit	59
Observability	60
Monitoring	60
Logging	60
Relationship diagram	60
Quick comparison	60
Part 7: Multi-Tenancy	62
What multi-tenancy is	62
Tenant isolation models	62
Shared database, shared schema	62
Shared database, separate schemas	62
Separate database per tenant	62
Separate deployment per tenant	63
Hybrid model	63
Security implications	63
Performance implications	63
Where tenancy support belongs	64
Part 8: Internationalization and Localization	65
i18n	65
l10n	65
Translations	65
Locale management	65
Currency handling	65
Timezone handling	66
Regional formatting	66
Responsibility split	66

Part 9: Building Kanbien v1	67
Must Have on day one	67
config	67
logging	67
validation	67
authn	67
authz	67
tenancy, lightweight	67
persistence	67
security	67
audit	67
events, minimal	68
Should Have as the platform grows	68
Enterprise Future	68
Part 10: Final Architecture Review	69
Ideal packages/core folder structure	69
Why each folder exists and what consumes it	72
How core connects to apps, platform, harness, and infrastructure	73
Most common mistakes founders make	74
Most common mistakes engineers make	74
Principles to guide Kanbien for the next 10 years	74

Part 1: Foundations

What a shared core package is

A shared core package is an internal library that holds the foundational concepts and contracts used across many parts of a platform. In Kanbien, that package would live at:

```
kanbien/  
  packages/  
    core/
```

It is not a product application. It is not a feature. It is not the cloud platform itself. It is the shared layer that gives the rest of Kanbien a consistent language for identity, permissions, tenancy, validation, events, queues, logging, audit, configuration, files, notifications, and similar cross-cutting concerns.

A useful analogy is a city. Individual buildings are applications. Roads, addresses, water standards, fire codes, and electrical rules are the shared foundations. `packages/core` is closer to those foundations than to any one building. It says, “This is how Kanbien names a tenant, records an audit event, represents a user principal, emits an event, validates an error, or describes a file.” It does not decide every workflow in every app.

Why shared core packages exist

Shared core packages exist because serious software platforms repeatedly need the same answers:

```
Who is this user?  
What tenant are they acting inside?  
What are they allowed to do?  
How do we validate this input?  
How do we record an audit trail?  
How do we publish an event?  
How do we store files?  
How do we log and monitor failures?  
How do we send notifications?  
How do we load configuration?
```

Without a shared core, every app invents its own answer. That seems fast early on, but it creates long-term inconsistency. One app checks permissions in one way. Another forgets tenant isolation. Another logs tokens. Another emits events without versioning. Another stores local time instead of UTC. The result is not just messy architecture; it is operational and security risk.

What problems shared core packages solve

A good shared core solves four large problems.

First, it creates consistency. Every part of Kanbien can use the same concepts for `TenantId`, `Principal`, `Permission`, `AuditEvent`, `EventEnvelope`, `Locale`, and `CorrelationId`.

Second, it enables reuse. Common behaviours such as validation errors, audit recording, tenant context, logging fields, or event metadata are implemented once and used many times.

Third, it supports governance. Enterprise-grade concerns such as security, compliance, audit, data retention, and observability become built into the platform rather than handled as afterthoughts.

Fourth, it improves delivery speed. New apps and services can start with a known foundation instead of debating basic mechanics every time.

The real-world analogy is a restaurant group. Each restaurant may have its own menu, but food safety, accounting, supply-chain coding, allergy warnings, and staff access rules should not be reinvented per location.

What problems shared core packages create when used incorrectly

A shared core package can become dangerous when it becomes the place for “anything shared.” That is the path to a dumping ground.

The biggest failure modes are:

- **Hidden coupling:** core imports application code, so supposedly stable foundations now depend on volatile features.
- **Over-abstraction:** the team creates generic frameworks before real product needs exist.
- **Blast radius:** a core bug breaks many apps.
- **Versioning bottlenecks:** every small change to core requires coordination across many consumers.
- **Ownership ambiguity:** everyone uses core, but no one feels responsible for it.

The junk-drawer analogy is accurate. At first, a drawer for spare cables and tools feels convenient. Later, no one knows what is safe, current, or important. Core needs clear admission rules.

How core differs from applications, features, platform, infrastructure, design systems, and utilities

Applications are deployable products or services, such as an admin app, API app, customer portal, or worker service. They own product-specific workflows.

Features are user-facing or domain capabilities, such as onboarding, billing, search, document review, or task assignment. They answer, “What does the product do?”

Platform is the runtime implementation layer. Core may define `FileStorage`; platform provides `S3FileStorage`. Core may define `EventBus`; platform implements it using Kafka, SQS, RabbitMQ, or an in-memory bus.

Infrastructure is the actual provisioned substrate: databases, networks, queues, buckets, IAM roles, load balancers, DNS, secrets managers, and deployment targets.

Design systems are shared UI foundations: typography, buttons, form fields, layout primitives, tokens, and accessible components. They render concepts to users; core defines many of the underlying concepts.

Utility libraries are generic helper collections. A date helper, string helper, or array helper can be useful, but it is not automatically core. Core should contain strategic Kanban concepts, not miscellaneous convenience functions.

Why enterprise teams create a shared core layer

Enterprise teams create a shared core because they need more than working features. They need repeatability, auditability, security, operational consistency, and governance across many teams and products.

At small scale, inconsistent logging is annoying. At enterprise scale, inconsistent logging slows incident response. At small scale, weak audit is inconvenient. At enterprise scale, weak audit can lose a customer. At small scale, tenant leakage is embarrassing. At enterprise scale, it can be existential.

Enterprise shared core layers usually standardize:

- identity and sessions
- permissions and policy decisions
- tenant context and isolation
- audit records
- event envelopes
- validation error shapes
- structured logs and metrics
- configuration and secret references
- file storage contracts

- notification contracts
- localization concepts

When a shared core layer becomes a dumping ground and how to prevent it

Core becomes a dumping ground when the admission rule is merely, “more than one place might use this.” That is not enough.

Use this rule instead:

A module belongs in core only if it defines a stable, cross-cutting Kanbien concept or a contract that multiple parts of the system must use consistently.

A practical checklist:

- Is it product-specific? Keep it out of core.
- Does it import an app? Keep it out of core.
- Does it import a cloud SDK or ORM? Usually put that implementation in platform.
- Is the concept stable enough? If not, keep it local.
- Does adding it reduce inconsistency or risk? If not, keep it local.
- Is there an owner? If not, do not add it.

The safest sequence is: start local, observe repetition, extract a stable abstraction, document it, test it, then move it to core.

Part 2: Understanding the Kanbien Architecture

Kanbien is assumed to have this top-level structure:

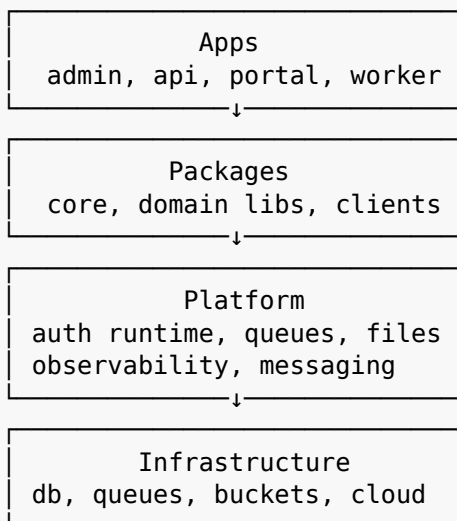
```
kanbien/  
  apps/  
  packages/  
  platform/  
  infra/  
  harness/  
  tools/  
  docs/
```

A practical interpretation is:

apps/	Deployable products and services
packages/	Shared libraries and domain packages
platform/	Runtime implementations and internal platform services
infra/	Infrastructure as code and cloud resources
harness/	Local, test, and development orchestration
tools/	Build, codegen, lint, release, and migration tooling
docs/	Architecture and operational documentation

packages/core sits inside packages/ and provides the stable foundation used by apps, platform implementations, harnesses, and sometimes tools.

Conceptual stack



The important conceptual flow is:

```
Apps  
↓  
Packages  
↓  
Platform  
↓  
Infrastructure
```

Apps use packages. Packages define reusable contracts and concepts. Platform implements shared runtime services. Infrastructure provisions the actual resources.

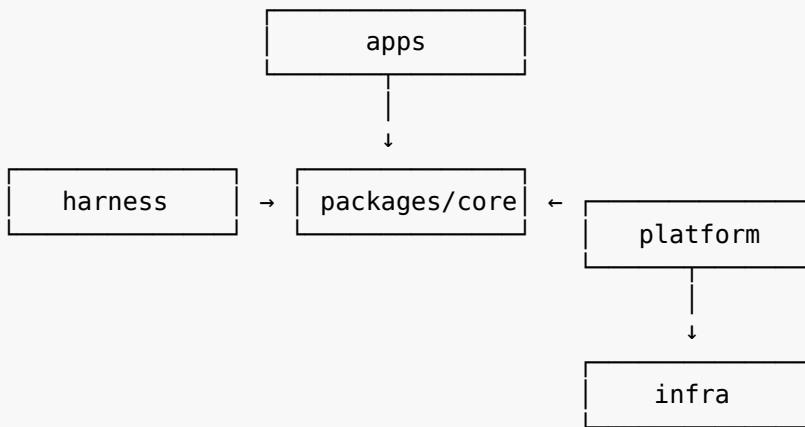
Code dependency direction

Conceptually, apps sit above packages, platform, and infrastructure. In code, the cleanest rule is:

```
apps      → packages/core
platform  → packages/core
harness   → packages/core + platform test implementations
tools     → packages/core only when needed
```

```
packages/core -X→ apps
packages/core -X→ platform
packages/core -X→ infra
```

Another way to view it:



What depends on what

Apps may depend on packages/core, feature/domain packages, generated API clients, platform adapters, and the design system for frontend applications.

Platform may depend on packages/core, cloud SDKs, database clients, queue clients, and observability SDKs.

Infrastructure usually does not import runtime application code. It provisions resources such as databases, topics, queues, buckets, networks, IAM roles, and deployment targets.

Core should be conservative. It should avoid dependencies on apps, platform, infrastructure, large frameworks, web servers, ORMs, cloud SDKs, and vendor-specific clients.

What should never depend on what

Danger signs include:

```
packages/core importing apps/*
packages/core importing infra/*
packages/core importing platform/*
apps importing raw infrastructure details directly
platform importing app-specific workflows
infra containing product business rules
```

Bad:

```
// packages/core/audit
import { CustomerPlan } from "../../apps/billing/domain";
```

Better:

```
// packages/core/audit
export interface AuditEvent {
  actorId: string;
  tenantId?: string;
  action: string;
  resourceType: string;
  resourceId: string;
  occurredAt: Date;
}
```

Why dependency direction matters

Things that change often should depend on things that change rarely. Product workflows, screens, and feature logic change frequently. Core contracts should change carefully. Infrastructure changes deliberately.

If core depends on apps, changing an app can break the foundation. That reverses stability and creates architectural decay.

How poor dependency management causes architectural decay

Decay often starts innocently:

```
We just need this helper from the app.
Now core needs the app's User type.
Now another app depends on that User type.
Now we cannot change users without breaking everything.
No one knows what owns what.
```

The cure is discipline:

```
Core defines ports.
Platform implements ports.
Apps compose ports into workflows.
Infrastructure provisions resources.
```

Part 3: Designing packages/core

At enterprise scale, packages/core should be organized around capability modules. Each module should expose types, interfaces, errors, policies, small pure helpers, and optionally in-memory or no-op test implementations.

It should not contain product-specific workflows or concrete cloud implementations.

The evaluated module set is:

async-jobs/, authn/, authz/, notifications/, validation/, persistence/, files/, events/, queues/, logging/, monitoring/, audit/, reporting/, analytics/, config/, tenancy/, security/, i18n/, localization/

A good internal shape for most modules is:

```
module/  
  index.ts  
  types.ts  
  errors.ts  
  ports.ts  
  policy.ts  
  testing.ts
```

Not every module needs every file. Avoid empty ceremony. Use the shape when it clarifies ownership.

async-jobs/

Purpose. Defines the common shape and lifecycle of work that should happen outside the immediate request-response path.

Real-world analogy. A restaurant waiter takes the order immediately, while the kitchen prepares the meal away from the table.

Responsibilities.

- job definitions and payload contracts
- job handler interfaces
- idempotency keys
- retry and scheduling concepts
- job status and result types

Example use cases.

- send welcome email
- generate a report
- process an uploaded CSV
- sync to an external CRM
- purge expired sessions

What belongs there.

- Job, JobId, JobHandler, JobScheduler interfaces
- retry policy types
- scheduled job contracts
- idempotency metadata

What does not belong there.

- the concrete worker process
- queue-provider SDK code
- product-specific job handlers
- cron infrastructure definitions

Typical APIs.

- Job
- JobHandler
- JobScheduler.enqueue()
- JobScheduler.schedule()

Common mistakes.

- running slow work inside API requests
- no idempotency
- no DLQ path
- unbounded retries
- placing app-specific jobs in core

Enterprise considerations.

- tenant-aware processing
- back-pressure
- priority queues
- DLQ operations
- job observability
- auditing for sensitive jobs

authn/

Purpose. Establishes who or what is making a request.

Real-world analogy. Authentication is showing a passport at the airport.

Responsibilities.

- principal and session concepts
- token/session/API-key verification interfaces
- service identity support
- authentication errors
- current principal context

Example use cases.

- verify a JWT
- read a session cookie
- authenticate a service account
- identify the current user

What belongs there.

- Principal type
- Authenticator interface
- AuthenticationInput
- UnauthenticatedError

What does not belong there.

- Auth0/Cognito-specific SDK code
- login UI
- password reset screens
- authorization decisions

Typical APIs.

- Authenticator.authenticate()
- Principal
- requirePrincipal()

Common mistakes.

- confusing authn with authz

- trusting unverified tokens
- not supporting service identities
- putting permissions into identity verification

Enterprise considerations.

- SSO
- MFA
- SAML/OIDC
- SCIM
- service accounts
- session revocation
- privileged access logging

authz/

Purpose. Determines whether a principal is allowed to perform an action on a resource.

Real-world analogy. Authentication gets you into the building; authorization decides which rooms you may enter.

Responsibilities.

- permission vocabulary
- role and policy types
- authorization request/decision model
- access denied errors
- resource/action conventions

Example use cases.

- can a user invite another user?
- can a service delete a file?
- can an admin view billing?
- can a support agent impersonate a customer?

What belongs there.

- Authorizer interface
- AuthorizationRequest
- AuthorizationDecision
- Permission types

What does not belong there.

- all product-specific permission definitions
- UI-only visibility logic
- database-specific permission queries
- business workflow code

Typical APIs.

- authorize()
- requireAllowed()
- AuthorizationDecision

Common mistakes.

- checking permissions only in the UI
- scattering checks inconsistently
- using roles for everything
- ignoring tenant boundaries

Enterprise considerations.

- RBAC

- ABAC
- delegated administration
- custom roles
- policy auditability
- separation of duties

notifications/

Purpose. Defines how Kanbien sends messages to users or external systems without binding app code to a provider.

Real-world analogy. Notifications are the organisation's mailroom.

Responsibilities.

- notification contracts
- recipient model
- template references
- delivery status
- preference concepts

Example use cases.

- welcome email
- password reset
- report ready alert
- security alert
- workflow assignment

What belongs there.

- Notification
- NotificationRecipient
- NotificationService
- NotificationReceipt

What does not belong there.

- SendGrid/Twilio/provider implementation
- marketing campaign logic
- product-specific copy
- UI inbox screens

Typical APIs.

- NotificationService.send()
- NotificationChannel
- NotificationReceipt

Common mistakes.

- sending email directly from business logic
- no delivery tracking
- no preferences
- no localization
- no audit for sensitive messages

Enterprise considerations.

- consent
- unsubscribe
- deliverability
- template approval
- PII controls
- rate limits

- localized templates

validation/

Purpose. Standardizes how Kanbien accepts, rejects, and explains bad input.

Real-world analogy. Validation is the bouncer checking that input is acceptable before it enters the system.

Responsibilities.

- validation result model
- validation error format
- common validators
- input normalization helpers
- schema wrapper conventions

Example use cases.

- validate email
- validate UUID
- validate uploaded file metadata
- validate API payloads
- normalize strings

What belongs there.

- ValidationResult
- ValidationError
- common low-level validators
- API error shaping helpers

What does not belong there.

- every app-specific form schema
- authorization logic
- database constraints only
- UI-only validation

Typical APIs.

- ValidationResult
- ValidationError
- requiredString()
- isEmail()

Common mistakes.

- only validating on the frontend
- vague errors
- mixing validation with permissions
- different error shapes per app

Enterprise considerations.

- localized messages
- contract testing
- PII-safe errors
- consistent API errors
- schema versioning

persistence/

Purpose. Defines common data-access patterns without coupling core to a specific database or ORM.

Real-world analogy. Core defines what a filing cabinet must do; platform chooses the actual cabinet.

Responsibilities.

- transaction interfaces
- pagination types
- repository conventions
- optimistic locking concepts
- outbox contracts

Example use cases.

- run a transaction
- paginate records
- store an outbox event
- apply optimistic concurrency
- standardize repository shape

What belongs there.

- TransactionManager
- TransactionContext
- PageRequest
- Page
- Outbox interfaces

What does not belong there.

- Prisma models
- SQL queries
- Mongo-specific operations
- app-specific schema and migrations

Typical APIs.

- TransactionManager.run()
- PageRequest
- Repository conventions
- Outbox.add()

Common mistakes.

- putting ORM models in core
- leaking DB-specific types
- no transaction boundaries
- publishing events outside the DB transaction

Enterprise considerations.

- data consistency
- migrations
- retention
- tenant partitioning
- read replicas
- data residency
- encryption

files/

Purpose. Defines how binary objects and their metadata are stored, retrieved, protected, and deleted.

Real-world analogy. Files are the warehouse; the database stores the inventory card, not usually the physical box.

Responsibilities.

- file metadata model
- storage interface
- content type and size policy concepts
- access contracts
- retention hooks

Example use cases.

- upload PDF
- store generated report
- attach evidence
- download export
- store profile image

What belongs there.

- FileStorage interface
- PutFileInput
- StoredFile
- FileObject
- file access types

What does not belong there.

- S3/GCS/Azure SDK code
- image-processing implementation
- product-specific document workflows
- virus scanner implementation

Typical APIs.

- FileStorage.put()
- FileStorage.get()
- FileStorage.delete()

Common mistakes.

- public buckets by accident
- no content-type validation
- no access audit
- large files in DB rows
- no retention model

Enterprise considerations.

- signed URLs
- encryption
- virus scanning
- legal hold
- tenant isolation
- data residency
- access logging

events/

Purpose. Defines how meaningful business events are represented and published.

Real-world analogy. Events are the organisation's newspaper: they say what happened.

Responsibilities.

- DomainEvent type
- EventEnvelope
- EventBus contract
- correlation/causation IDs

- event versioning rules

Example use cases.

- UserCreated
- FileUploaded
- ReportGenerated
- TenantProvisioned
- RoleChanged

What belongs there.

- DomainEvent
- EventEnvelope
- EventBus
- event metadata and versioning conventions

What does not belong there.

- Kafka/SNS implementation
- all product event definitions
- event-triggered business workflows
- consumer deployments

Typical APIs.

- EventBus.publish()
- EventEnvelope
- DomainEvent

Common mistakes.

- using events as commands
- no schema versioning
- no idempotency
- no correlation IDs
- assuming exactly-once delivery

Enterprise considerations.

- schema governance
- PII control
- event replay
- retention
- lineage
- consumer ownership
- cross-region delivery

queues/

Purpose. Defines message queues used to buffer and retry background work.

Real-world analogy. A queue is a line at the post office; work waits until a worker is ready.

Responsibilities.

- queue interface
- message envelope
- consumer contract
- retry metadata
- dead-letter concepts

Example use cases.

- send email later
- process upload

- generate report
- retry webhook
- sync external system

What belongs there.

- Queue
- QueueMessage
- QueueHandler
- QueueSendOptions
- DLQ metadata

What does not belong there.

- SQS/RabbitMQ/Redis implementation
- worker deployment scripts
- product-specific handlers
- cloud IAM rules

Typical APIs.

- Queue.send()
- Queue.consume()
- QueueHandler

Common mistakes.

- no idempotency
- no DLQ
- no retry backoff
- no monitoring of queue depth
- ignoring poison messages

Enterprise considerations.

- throughput
- back-pressure
- ordering
- tenant fairness
- replay
- encryption
- regional isolation

logging/

Purpose. Standardizes structured operational logging across apps and platform services.

Real-world analogy. Logs are the diary of the software system.

Responsibilities.

- Logger interface
- log levels
- structured fields
- context propagation
- redaction helpers

Example use cases.

- request started
- job failed
- permission denied
- external API timeout
- report generated

What belongs there.

- Logger
- LoggerFactory
- LogFields
- redaction helpers
- correlation ID conventions

What does not belong there.

- Datadog/CloudWatch implementation
- app-specific messages
- raw secrets in logs
- audit storage

Typical APIs.

- `logger.info()`
- `logger.warn()`
- `logger.error()`
- `loggerFactory.child()`

Common mistakes.

- plain string logs only
- no correlation ID
- logging tokens
- too much noise
- no structured fields

Enterprise considerations.

- PII redaction
- retention
- searchability
- incident response
- cost control
- sampling

monitoring/

Purpose. Defines health checks and metrics that show whether systems are healthy and meeting expectations.

Real-world analogy. Monitoring is the cockpit dashboard that shows whether the aircraft is healthy.

Responsibilities.

- metrics interface
- health checks
- readiness/liveness types
- counters/gauges/timers
- basic SLI conventions

Example use cases.

- API latency
- queue depth
- job failure rate
- notification delivery rate
- database health

What belongs there.

- Metrics

- HealthCheck
- Timer helpers
- health status types

What does not belong there.

- Prometheus/Datadog implementation
- cloud alarms
- full observability backend
- app-specific dashboards

Typical APIs.

- metrics.increment()
- metrics.gauge()
- metrics.timing()
- healthCheck.check()

Common mistakes.

- adding monitoring only after incidents
- no alerts
- too many noisy alerts
- only measuring infrastructure
- no SLO thinking

Enterprise considerations.

- SLIs/SLOs
- alert routing
- incident response
- capacity planning
- customer-facing uptime commitments

audit/

Purpose. Creates a durable accountability record of significant user and system actions.

Real-world analogy. Audit is CCTV plus access logs for important business actions.

Responsibilities.

- audit event model
- actor/resource/action model
- change tracking
- AuditService interface
- audit error types

Example use cases.

- role changed
- file downloaded
- user invited
- data exported
- tenant setting changed

What belongs there.

- AuditEvent
- AuditActor
- AuditResource
- AuditChange
- AuditService

What does not belong there.

- debug logs
- analytics events
- product-specific audit screens
- database-specific storage

Typical APIs.

- audit.record()
- AuditEvent
- AuditChange

Common mistakes.

- confusing logs with audit
- missing actor
- missing tenant
- mutable records
- no retention policy

Enterprise considerations.

- tamper resistance
- legal discovery
- access controls
- exportability
- privileged action review
- long retention

reporting/

Purpose. Defines repeatable report generation, export formats, and report lifecycle concepts.

Real-world analogy. Reporting is the official report pack prepared for a management or customer review.

Responsibilities.

- report request model
- report generator interface
- formats
- generated report metadata
- export status concepts

Example use cases.

- monthly usage report
- CSV export
- compliance report
- invoice summary
- user activity report

What belongs there.

- ReportRequest
- ReportGenerator
- GeneratedReport
- ReportFormat

What does not belong there.

- every product-specific report query
- BI dashboards
- analytics taxonomy
- heavy warehouse implementation

Typical APIs.

- ReportGenerator.generate()
- ReportRequest
- GeneratedReport

Common mistakes.

- running heavy reports on write DB
- no export audit
- no access control
- confusing reporting with analytics

Enterprise considerations.

- async generation
- large exports
- read replicas/warehouse
- PII handling
- retention
- tenant filtering

analytics/

Purpose. Defines product and behavioural tracking without binding apps to a vendor.

Real-world analogy. Analytics is the product instrument panel showing how people behave.

Responsibilities.

- analytics event model
- track interface
- identify interface
- traits
- consent-aware concepts
- taxonomy conventions

Example use cases.

- signup completed
- feature used
- invite accepted
- report generated
- trial converted

What belongs there.

- AnalyticsEvent
- AnalyticsIdentity
- Analytics interface
- consent/taxonomy types

What does not belong there.

- audit evidence
- BI dashboards
- vendor-specific SDKs
- funnel definitions tightly coupled to one app

Typical APIs.

- analytics.track()
- analytics.identify()
- AnalyticsEvent

Common mistakes.

- using analytics for audit
- tracking unnecessary PII
- no naming governance
- ignoring consent

Enterprise considerations.

- privacy
- consent
- data minimization
- event taxonomy
- regional rules
- retention
- customer isolation

config/

Purpose. Defines typed configuration access and startup validation.

Real-world analogy. Configuration is the control panel, not the machine itself.

Responsibilities.

- ConfigProvider interface
- environment names
- required key validation
- secret reference concepts
- feature flag access patterns

Example use cases.

- load auth issuer
- read log level
- resolve queue names
- select region
- read feature flag

What belongs there.

- ConfigProvider
- EnvironmentName
- requireConfig()
- config errors

What does not belong there.

- actual secrets
- cloud secret manager implementation
- feature-specific business constants
- hardcoded environment hacks

Typical APIs.

- getString()
- getOptionalString()
- getBoolean()
- requireConfig()

Common mistakes.

- scattering process.env
- no startup validation
- checking secrets into code
- mixing config with business rules

Enterprise considerations.

- secret rotation
- environment parity
- feature flag governance
- regional config
- change audit

tenancy/

Purpose. Defines tenant identity, tenant context, and isolation concepts.

Real-world analogy. An apartment building has shared infrastructure, but each tenant's apartment must remain private.

Responsibilities.

- TenantId type
- TenantContext
- TenantResolver interface
- tenant propagation
- tenant isolation errors

Example use cases.

- resolve tenant from hostname
- scope database queries
- attach tenant to events
- isolate files
- tenant-aware audit

What belongs there.

- TenantId
- TenantContext
- TenantResolver
- requireTenant()

What does not belong there.

- tenant billing plan workflows
- tenant admin UI
- dedicated DB provisioning code
- customer onboarding screens

Typical APIs.

- TenantResolver.resolve()
- requireTenant()
- TenantContext

Common mistakes.

- trusting tenant ID from request body
- forgetting tenant_id filters
- not propagating tenant context
- mixing tenant with billing account

Enterprise considerations.

- data isolation
- data residency
- tenant-level audit
- noisy-neighbor control
- dedicated infrastructure for large customers

security/

Purpose. Provides common defensive primitives and contracts.

Real-world analogy. Security is the locks, alarms, safe handling rules, and inspection routines.

Responsibilities.

- redaction
- sensitive value handling
- cryptographic service interfaces
- rate-limit contracts
- security error types

Example use cases.

- redact logs
- hash a secret
- generate token
- classify sensitive fields
- rate-limit login attempts

What belongs there.

- redactObject()
- SecureRandom
- SecretHasher
- RateLimiter interfaces
- security constants

What does not belong there.

- actual secrets
- home-grown cryptography
- KMS implementation
- app-specific security exceptions

Typical APIs.

- redactObject()
- SecretHasher.hash()
- SecretHasher.verify()
- SecureRandom.token()

Common mistakes.

- inventing cryptography
- logging secrets
- no input size limits
- no secure defaults
- no key rotation path

Enterprise considerations.

- threat modeling
- key rotation
- secrets management
- PII classification
- secure SDLC
- vulnerability response

i18n/

Purpose. Defines translation-ready mechanisms: locale, keys, catalogs, translators, fallbacks, and interpolation.

Real-world analogy. Internationalization is designing a building so signs can later be changed into different languages without rebuilding the walls.

Responsibilities.

- Locale type
- Translator interface
- message catalog contract
- fallback strategy
- translation params

Example use cases.

- translate validation errors
- translate notification subjects
- resolve fallback language
- localize product messages

What belongs there.

- Locale
- Translator
- MessageCatalog
- fallback rules

What does not belong there.

- all product copy
- UI components
- marketing copy workflow
- country-specific legal policy

Typical APIs.

- translator.t()
- TranslationParams
- Locale

Common mistakes.

- hardcoding English strings
- concatenating translated strings
- no fallback locale
- no pluralization strategy

Enterprise considerations.

- translation workflow
- pluralization
- right-to-left support
- localized notifications
- regulatory language requirements

Localization/

Purpose. Handles region-specific formatting and behaviour such as date, time, number, currency, and timezone handling.

Real-world analogy. Internationalization makes the car sellable globally; localization adapts the steering wheel, dashboard units, and rules for each market.

Responsibilities.

- locale parsing
- timezone formatting
- currency formatting

- date/time normalization
- regional formatting interfaces

Example use cases.

- format money
- display time in user timezone
- format date for notification
- store UTC timestamps
- regional number formatting

What belongs there.

- LocalizationService
- locale utilities
- timezone helpers
- currency formatting primitives

What does not belong there.

- product copy
- UI components
- pricing strategy
- country-specific legal advice

Typical APIs.

- formatDate()
- formatDateTime()
- formatCurrency()

Common mistakes.

- storing local time as canonical
- assuming one currency
- assuming one date format
- ignoring daylight saving time

Enterprise considerations.

- regional compliance
- timezone correctness
- currency precision
- tax/invoice localization
- multi-region operations

Part 4: Deep Dive into Every Module

This section expands each module from first principles: why it exists, how it works internally, example TypeScript, example Kanbien usage, and enterprise concerns. The examples are intentionally small. In production, the module should also include tests, documentation, and production implementations in platform/ where appropriate.

async-jobs/ deep dive

Why it exists

It keeps user-facing APIs fast and makes slow or unreliable work retryable and observable.

How it works internally

- A job has a type, payload, tenant context, idempotency key, attempt count, and timestamps.
- A scheduler enqueues or schedules jobs.
- A worker runtime finds the right handler and executes it.
- Failures are retried according to policy and eventually moved to a dead-letter path.

Example implementation

```
export type JobId = string;

export interface Job<TPayload = unknown> {
  id: JobId;
  type: string;
  payload: TPayload;
  tenantId?: string;
  idempotencyKey?: string;
  attempts: number;
  createdAt: Date;
}

export interface JobContext {
  correlationId: string;
  now: Date;
}

export interface JobHandler<TPayload = unknown> {
  type: string;
  handle(job: Job<TPayload>, context: JobContext): Promise<void>;
}

export interface JobScheduler {
  enqueue<TPayload>({
    type: string,
    payload: TPayload,
    options?: { tenantId?: string; idempotencyKey?: string; runAt?: Date }
  }): Promise<JobId>;
}
```

Example Kanbien usage

```
await jobs.enqueue(
  "notifications.sendWelcomeEmail",
```

```
{ userId: user.id, email: user.email },
{ tenantId, idempotencyKey: `welcome-email:${user.id}` }
);
```

Enterprise concerns

- tenant-aware processing
- back-pressure
- priority queues
- DLQ operations
- job observability
- auditing for sensitive jobs

Architectural warning

Keep the core module focused on stable contracts and cross-cutting primitives. Put concrete provider integrations in platform/. Put product-specific behaviour in apps/ or feature packages.

authn/ deep dive

Why it exists

Every secure workflow needs a trusted identity before it can evaluate permissions or produce audit records.

How it works internally

- A request arrives with a token, API key, or session.
- An authenticator verifies the credential.
- A Principal object is created.
- The Principal is carried in request context.

Example implementation

```
export type PrincipalType = "user" | "service";

export interface Principal {
  subject: string;
  type: PrincipalType;
  email?: string;
  displayName?: string;
  tenantIds?: string[];
  scopes?: string[];
}

export interface AuthenticationInput {
  authorizationHeader?: string;
  apiKey?: string;
  sessionCookie?: string;
}

export interface Authenticator {
  authenticate(input: AuthenticationInput): Promise<Principal>;
}

export class UnauthenticatedError extends Error {
```



```

constructor(message = "Unauthenticated") {
  super(message);
  this.name = "UnauthenticatedError";
}
}

```

Example Kanbien usage

```

const principal = await authenticator.authenticate({
  authorizationHeader: request.headers.authorization,
});

requestContext.principal = principal;

```

Enterprise concerns

- SSO
- MFA
- SAML/OIDC
- SCIM
- service accounts
- session revocation
- privileged access logging

Architectural warning

Keep the core module focused on stable contracts and cross-cutting primitives. Put concrete provider integrations in `platform/`. Put product-specific behaviour in `apps/` or feature packages.

authz/ deep dive

Why it exists

Identity does not imply permission. Authorization is the control point that protects resources and tenants.

How it works internally

- The system builds an authorization request containing actor, tenant, action, resource, and context.
- A policy engine or role mapping evaluates it.
- The result is allow or deny, with an optional reason.
- Apps enforce the decision before state changes.

Example implementation

```

export type AuthorizationEffect = "allow" | "deny";

export interface AuthorizationRequest {
  principalId: string;
  tenantId?: string;
  action: string;
  resource: {
    type: string;
    id?: string;
  };
}

```

```

    attributes?: Record<string, unknown>;
  };
}

export interface AuthorizationDecision {
  effect: AuthorizationEffect;
  reason?: string;
}

export interface Authorizer {
  authorize(request: AuthorizationRequest): Promise<AuthorizationDecision>;
}

export async function requireAllowed(
  authorizer: Authorizer,
  request: AuthorizationRequest
): Promise<void> {
  const decision = await authorizer.authorize(request);
  if (decision.effect !== "allow") throw new Error(decision.reason ?? "Access denied");
}

```

Example Kanbien usage

```

await requireAllowed(authorizer, {
  principalId: principal.subject,
  tenantId,
  action: "user.create",
  resource: { type: "user" },
});

```

Enterprise concerns

- RBAC
- ABAC
- delegated administration
- custom roles
- policy auditability
- separation of duties

Architectural warning

Keep the core module focused on stable contracts and cross-cutting primitives. Put concrete provider integrations in `platform/`. Put product-specific behaviour in `apps/` or feature packages.

notifications/ deep dive

Why it exists

Messaging is cross-cutting, failure-prone, and often regulated by consent and security rules.

How it works internally

- An app creates a notification request.
- A template renderer combines template and data.
- A provider adapter delivers through email, SMS, push, in-app, or webhook.
- Delivery results are tracked and failures retried when appropriate.

Example implementation

```
export type NotificationChannel = "email" | "sms" | "push" | "in_app" | "webhook";

export interface NotificationRecipient {
  userId?: string;
  email?: string;
  phone?: string;
}

export interface Notification {
  type: string;
  channel: NotificationChannel;
  recipient: NotificationRecipient;
  template: string;
  data: Record<string, unknown>;
  tenantId?: string;
  locale?: string;
}

export interface NotificationReceipt {
  id: string;
  status: "accepted" | "rejected";
  providerMessageId?: string;
}

export interface NotificationService {
  send(notification: Notification): Promise<NotificationReceipt>;
}
```

Example Kanbien usage

```
await notifications.send({
  type: "user.welcome",
  channel: "email",
  recipient: { userId: user.id, email: user.email },
  template: "welcome_user",
  data: { name: user.name },
  tenantId,
  locale: user.locale,
});
```

Enterprise concerns

- consent
- unsubscribe
- deliverability
- template approval
- PII controls
- rate limits
- localized templates

Architectural warning

Keep the core module focused on stable contracts and cross-cutting primitives. Put concrete provider integrations in `platform/`. Put product-specific behaviour in `apps/` or feature packages.

validation/ deep dive

Why it exists

Bad input causes bugs, security issues, and inconsistent user experiences.

How it works internally

- Validators transform unknown input into known typed values.
- Failures return structured error codes and paths.
- Apps compose core validators with feature-specific schemas.
- API handlers map validation failures to a consistent response.

Example implementation

```
export interface ValidationError {
  path: string;
  code: string;
  message: string;
}

export type ValidationResult<T> =
  | { ok: true; value: T }
  | { ok: false; errors: ValidationError[] };

export function requiredString(path: string, value: unknown): ValidationResult<string> {
  if (typeof value !== "string" || value.trim() === "") {
    return { ok: false, errors: [{ path, code: "required", message: "Required" }] };
  }
  return { ok: true, value: value.trim() };
}

export function isEmail(value: string): boolean {
  return /^[^s@]+@[^s@]+\.[^s@]+$/ .test(value);
}
```

Example Kanbien usage

```
const email = requiredString("email", body.email);
if (!email.ok) return badRequest(email.errors);
if (!isEmail(email.value)) {
  return badRequest([{ path: "email", code: "invalid_email", message: "Invalid email
    ↪ address" }]);
}
```

Enterprise concerns

- localized messages
- contract testing
- PII-safe errors
- consistent API errors
- schema versioning

Architectural warning

Keep the core module focused on stable contracts and cross-cutting primitives. Put concrete provider integrations in platform/. Put product-specific behaviour in apps/ or feature packages.

persistence/ deep dive

Why it exists

Data access is central to correctness, but the core package should remain database-agnostic.

How it works internally

- Transactions define an atomic boundary.
- Repositories hide storage details from use cases.
- Pagination types standardize list endpoints.
- Outbox records make state changes and event publication reliable.

Example implementation

```
export interface TransactionContext {
  id: string;
}

export interface TransactionManager {
  run<T>(work: (tx: TransactionContext) => Promise<T>): Promise<T>;
}

export interface PageRequest {
  limit: number;
  cursor?: string;
}

export interface Page<T> {
  items: T[];
  nextCursor?: string;
}

export interface VersionedEntity {
  version: number;
}
```

Example Kanbien usage

```
await transactionManager.run(async (tx) => {
  const user = await userRepository.create({ tenantId, email, name }, tx);
  await outbox.add({ type: "UserCreated", version: 1, payload: { userId: user.id } }, tx);
  return user;
});
```

Enterprise concerns

- data consistency
- migrations
- retention
- tenant partitioning
- read replicas
- data residency
- encryption

Architectural warning

Keep the core module focused on stable contracts and cross-cutting primitives. Put concrete provider integrations in `platform/`. Put product-specific behaviour in `apps/` or feature packages.

files/ deep dive

Why it exists

Files are high-risk because they are large, user-controlled, and often contain sensitive data.

How it works internally

- Apps stream file bodies to storage.
- Metadata is returned and often stored in the application database.
- Access is mediated by authorization and signed URLs.
- Deletion and retention are policy-driven.

Example implementation

```
export interface PutFileInput {
  filename: string;
  contentType: string;
  sizeBytes: number;
  body: AsyncIterable<Uint8Array>;
  tenantId?: string;
  metadata?: Record<string, string>;
}

export interface StoredFile {
  id: string;
  filename: string;
  contentType: string;
  sizeBytes: number;
  tenantId?: string;
  createdAt: Date;
}

export interface FileObject extends StoredFile {
  body: AsyncIterable<Uint8Array>;
}

export interface FileStorage {
  put(input: PutFileInput): Promise<StoredFile>;
  get(id: string): Promise<FileObject>;
  delete(id: string): Promise<void>;
}
```

Example Kanbien usage

```
const stored = await fileStorage.put({
  filename: upload.filename,
  contentType: upload.contentType,
  sizeBytes: upload.sizeBytes,
  body: upload.stream,
  tenantId,
  metadata: { uploadedBy: principal.subject },
});
```

```
});
```

Enterprise concerns

- signed URLs
- encryption
- virus scanning
- legal hold
- tenant isolation
- data residency
- access logging

Architectural warning

Keep the core module focused on stable contracts and cross-cutting primitives. Put concrete provider integrations in `platform/`. Put product-specific behaviour in `apps/` or feature packages.

events/ deep dive

Why it exists

Events decouple workflows and let multiple systems react to one business fact independently.

How it works internally

- A domain action creates an event.
- The event is wrapped in an envelope with metadata.
- The event bus publishes it to subscribers.
- Consumers handle it idempotently and independently.

Example implementation

```
export interface DomainEvent<TPayload = unknown> {
  type: string;
  version: number;
  payload: TPayload;
}

export interface EventEnvelope<TPayload = unknown> {
  id: string;
  event: DomainEvent<TPayload>;
  tenantId?: string;
  occurredAt: Date;
  correlationId?: string;
  causationId?: string;
  actorId?: string;
}

export interface EventBus {
  publish<TPayload>(envelope: EventEnvelope<TPayload>): Promise<void>;
}
```

Example Kanbien usage

```
await eventBus.publish({
  id: crypto.randomUUID(),
  event: { type: "UserCreated", version: 1, payload: { userId: user.id, email: user.email
↵ } },
  tenantId,
  actorId: principal.subject,
  occurredAt: new Date(),
  correlationId,
});
```

Enterprise concerns

- schema governance
- PII control
- event replay
- retention
- lineage
- consumer ownership
- cross-region delivery

Architectural warning

Keep the core module focused on stable contracts and cross-cutting primitives. Put concrete provider integrations in `platform/`. Put product-specific behaviour in `apps/` or feature packages.

queues/ deep dive

Why it exists

Queues let the platform absorb bursts, isolate failures, and retry work outside the request path.

How it works internally

- Producers send messages.
- The queue durably stores them.
- Consumers receive and process them.
- Failures are retried or moved to a dead-letter queue.

Example implementation

```
export interface QueueMessage<TPayload = unknown> {
  id: string;
  type: string;
  payload: TPayload;
  tenantId?: string;
  correlationId?: string;
  attempts: number;
}

export interface QueueSendOptions {
  delaySeconds?: number;
  idempotencyKey?: string;
}

export type QueueHandler<TPayload = unknown> = (message: QueueMessage<TPayload>) =>
↵ Promise<void>;
```



```
export interface Queue<TPayload = unknown> {
  send(message: Omit<QueueMessage<TPayload>, "id" | "attempts">, options?:
    ↳ QueueSendOptions): Promise<void>;
  consume(handler: QueueHandler<TPayload>): Promise<void>;
}
```

Example Kanbien usage

```
await queue.send({
  type: "send_welcome_email",
  tenantId,
  correlationId,
  payload: { userId: user.id, email: user.email },
});
```

Enterprise concerns

- throughput
- back-pressure
- ordering
- tenant fairness
- replay
- encryption
- regional isolation

Architectural warning

Keep the core module focused on stable contracts and cross-cutting primitives. Put concrete provider integrations in platform/. Put product-specific behaviour in apps/ or feature packages.

logging/ deep dive

Why it exists

When production breaks, structured logs are often the fastest route to understanding the execution path.

How it works internally

- A logger accepts a message and fields.
- A child logger carries context such as tenantId and correlationId.
- Platform serializes and ships logs to a destination.
- Security helpers redact sensitive fields before output.

Example implementation

```
export type LogFields = Record<string, string | number | boolean | null | undefined>;

export interface Logger {
  debug(message: string, fields?: LogFields): void;
  info(message: string, fields?: LogFields): void;
  warn(message: string, fields?: LogFields): void;
  error(message: string, fields?: LogFields & { error?: unknown }): void;
}
```

```
export interface LoggerFactory {
  child(fields: LogFields): Logger;
}
```

Example Kanbien usage

```
const log = loggerFactory.child({ tenantId, correlationId, actorId: principal.subject });
log.info("Creating user", { emailDomain: email.split("@")[1] });
```

Enterprise concerns

- PII redaction
- retention
- searchability
- incident response
- cost control
- sampling

Architectural warning

Keep the core module focused on stable contracts and cross-cutting primitives. Put concrete provider integrations in platform/. Put product-specific behaviour in apps/ or feature packages.

monitoring/ deep dive

Why it exists

Logs explain individual execution paths; monitoring tells you whether the system as a whole is healthy.

How it works internally

- Apps emit metrics around important paths.
- Health checks test dependencies.
- Platform exports signals to monitoring systems.
- Alerts fire when thresholds or SLOs are breached.

Example implementation

```
export interface Metrics {
  increment(name: string, tags?: Record<string, string>): void;
  gauge(name: string, value: number, tags?: Record<string, string>): void;
  timing(name: string, milliseconds: number, tags?: Record<string, string>): void;
}

export interface HealthCheck {
  name: string;
  check(): Promise<{ status: "healthy" | "degraded" | "unhealthy"; details?:
    ↳ Record<string, unknown> }>;
}
```

Example Kanbien usage

```
const started = Date.now();
try {
  await createUser(input);
  metrics.increment("user.create.success", { tenantId });
} finally {
  metrics.timing("user.create.duration_ms", Date.now() - started, { tenantId });
}
```

Enterprise concerns

- SLIs/SLOs
- alert routing
- incident response
- capacity planning
- customer-facing uptime commitments

Architectural warning

Keep the core module focused on stable contracts and cross-cutting primitives. Put concrete provider integrations in platform/. Put product-specific behaviour in apps/ or feature packages.

audit/ deep dive

Why it exists

Enterprise customers need evidence of who did what, when, to which resource, and under what tenant.

How it works internally

- Apps emit audit records at important control points.
- Records include actor, tenant, resource, action, timestamp, and optional changes.
- Platform writes records to durable append-oriented storage.
- Audit access is itself controlled and often audited.

Example implementation

```
export interface AuditActor {
  id: string;
  type: "user" | "service";
}

export interface AuditResource {
  type: string;
  id: string;
}

export interface AuditChange {
  field: string;
  before?: unknown;
  after?: unknown;
}

export interface AuditEvent {
  id?: string;
  action: string;
  actor: AuditActor;
```

```

tenantId?: string;
resource: AuditResource;
changes?: AuditChange[];
occurredAt: Date;
correlationId?: string;
ipAddress?: string;
}

export interface AuditService {
  record(event: AuditEvent): Promise<void>;
}

```

Example Kanbien usage

```

await audit.record({
  action: "user.created",
  actor: { id: principal.subject, type: "user" },
  tenantId,
  resource: { type: "user", id: user.id },
  occurredAt: new Date(),
  correlationId,
});

```

Enterprise concerns

- tamper resistance
- legal discovery
- access controls
- exportability
- privileged action review
- long retention

Architectural warning

Keep the core module focused on stable contracts and cross-cutting primitives. Put concrete provider integrations in platform/. Put product-specific behaviour in apps/ or feature packages.

reporting/ deep dive

Why it exists

Reports are often user-facing, auditable outputs and need consistent lifecycle, access, and export handling.

How it works internally

- A user requests a report with parameters and format.
- The app checks authorization.
- A generator builds the report, often asynchronously.
- The output is stored as a file and audited.

Example implementation

```

export type ReportFormat = "csv" | "pdf" | "xlsx" | "json";

```

```

export interface ReportRequest<TParams = Record<string, unknown>> {
  reportType: string;
  requestedBy: string;
  tenantId?: string;
  params: TParams;
  format: ReportFormat;
}

export interface GeneratedReport {
  id: string;
  format: ReportFormat;
  fileId: string;
  generatedAt: Date;
}

export interface ReportGenerator<TParams = Record<string, unknown>> {
  reportType: string;
  generate(request: ReportRequest<TParams>): Promise<GeneratedReport>;
}

```

Example Kanbien usage

```

const report = await usageReportGenerator.generate({
  reportType: "tenant_usage",
  requestedBy: principal.subject,
  tenantId,
  params: { from: "2026-01-01", to: "2026-01-31" },
  format: "csv",
});

```

Enterprise concerns

- async generation
- large exports
- read replicas/warehouse
- PII handling
- retention
- tenant filtering

Architectural warning

Keep the core module focused on stable contracts and cross-cutting primitives. Put concrete provider integrations in platform/. Put product-specific behaviour in apps/ or feature packages.

analytics/ deep dive

Why it exists

Product and leadership teams need behaviour signals, but those signals require governance and privacy discipline.

How it works internally

- Apps emit named events with properties.
- An adapter forwards them to a provider or warehouse.
- Identify calls attach traits to users or tenants.

- Consent and privacy rules filter or suppress events.

Example implementation

```
export interface AnalyticsEvent {
  name: string;
  userId?: string;
  tenantId?: string;
  properties?: Record<string, unknown>;
  occurredAt: Date;
}

export interface AnalyticsIdentity {
  userId: string;
  tenantId?: string;
  traits?: Record<string, unknown>;
}

export interface Analytics {
  track(event: AnalyticsEvent): Promise<void>;
  identify(identity: AnalyticsIdentity): Promise<void>;
}
```

Example Kanbien usage

```
await analytics.track({
  name: "user.created",
  userId: user.id,
  tenantId,
  properties: { source: "admin_invite" },
  occurredAt: new Date(),
});
```

Enterprise concerns

- privacy
- consent
- data minimization
- event taxonomy
- regional rules
- retention
- customer isolation

Architectural warning

Keep the core module focused on stable contracts and cross-cutting primitives. Put concrete provider integrations in platform/. Put product-specific behaviour in apps/ or feature packages.

config/ deep dive

Why it exists

Configuration drift and missing environment settings are common causes of deployment failures.

How it works internally

- Platform loads values from environment variables, files, or secret managers.
- Core exposes a typed provider interface.
- Apps validate required keys at startup.
- Secrets are referenced and fetched through controlled mechanisms.

Example implementation

```
export type EnvironmentName = "development" | "test" | "staging" | "production";

export interface ConfigProvider {
  getString(key: string): string;
  getOptionalString(key: string): string | undefined;
  getNumber(key: string): number;
  getBoolean(key: string): boolean;
}

export function requireConfig(config: ConfigProvider, key: string): string {
  const value = config.getOptionalString(key);
  if (!value) throw new Error(`Missing required config: ${key}`);
  return value;
}
```

Example Kanbien usage

```
const authIssuer = requireConfig(config, "AUTH_ISSUER");
const logLevel = config.getOptionalString("LOG_LEVEL") ?? "info";
```

Enterprise concerns

- secret rotation
- environment parity
- feature flag governance
- regional config
- change audit

Architectural warning

Keep the core module focused on stable contracts and cross-cutting primitives. Put concrete provider integrations in platform/. Put product-specific behaviour in apps/ or feature packages.

tenancy/ deep dive

Why it exists

Multi-tenancy is very expensive and risky to retrofit after data and workflows already exist.

How it works internally

- A request is mapped to a tenant from a trusted source.
- Membership and authorization are verified.
- TenantContext is carried through repositories, events, jobs, logs, audit, and analytics.
- Storage implementations enforce tenant boundaries.

Example implementation

```
export type TenantId = string;

export interface TenantContext {
  tenantId: TenantId;
  region?: string;
  locale?: string;
  timeZone?: string;
}

export interface TenantResolutionInput {
  hostname?: string;
  principalTenantIds?: string[];
  requestedTenantId?: string;
}

export interface TenantResolver {
  resolve(input: TenantResolutionInput): Promise<TenantContext>;
}

export function requireTenant(context?: TenantContext): TenantContext {
  if (!context) throw new Error("Tenant context required");
  return context;
}
```

Example Kanbien usage

```
const tenant = await tenantResolver.resolve({
  hostname: request.hostname,
  principalTenantIds: principal.tenantIds,
  requestedTenantId: request.params.tenantId,
});

await userRepository.create({ tenantId: tenant.tenantId, email });
```

Enterprise concerns

- data isolation
- data residency
- tenant-level audit
- noisy-neighbor control
- dedicated infrastructure for large customers

Architectural warning

Keep the core module focused on stable contracts and cross-cutting primitives. Put concrete provider integrations in platform/. Put product-specific behaviour in apps/ or feature packages.

security/ deep dive

Why it exists

Security helpers must be consistent, reviewed, and boring. Ad hoc security code becomes risk.

How it works internally

- Core exposes contracts and safe helpers.
- Platform binds those contracts to proven libraries or managed services.
- Apps call the contracts rather than handling secrets directly.
- Redaction runs before logs or error payloads leave the process.

Example implementation

```
const SENSITIVE_KEYS = ["password", "token", "authorization", "secret", "apiKey",  
  ↪ "cookie"];  
  
export function redactObject(input: Record<string, unknown>): Record<string, unknown> {  
  const output: Record<string, unknown> = {};  
  for (const [key, value] of Object.entries(input)) {  
    const isSensitive = SENSITIVE_KEYS.some((s) =>  
      ↪ key.toLowerCase().includes(s.toLowerCase()));  
    output[key] = isSensitive ? "[REDACTED]" : value;  
  }  
  return output;  
}  
  
export interface SecureRandom {  
  uuid(): string;  
  token(bytes: number): string;  
}  
  
export interface SecretHasher {  
  hash(value: string): Promise<string>;  
  verify(value: string, hash: string): Promise<boolean>;  
}
```

Example Kanbien usage

```
logger.info("Received webhook", redactObject({  
  provider: "example",  
  authorization: request.headers.authorization,  
  eventType: body.type,  
}));
```

Enterprise concerns

- threat modeling
- key rotation
- secrets management
- PII classification
- secure SDLC
- vulnerability response

Architectural warning

Keep the core module focused on stable contracts and cross-cutting primitives. Put concrete provider integrations in platform/. Put product-specific behaviour in apps/ or feature packages.

i18n/ deep dive

Why it exists

Hardcoded language assumptions become expensive once customers expect local language support.

How it works internally

- Apps refer to translation keys.
- Catalogs map keys to localized strings.
- Translator resolves locale with fallback.
- Parameters are interpolated safely.

Example implementation

```
export type Locale = string;

export interface TranslationParams {
  [key: string]: string | number | boolean | Date;
}

export interface Translator {
  t(key: string, params?: TranslationParams, locale?: Locale): string;
}

export class SimpleTranslator implements Translator {
  constructor(
    private readonly catalogs: Record<string, Record<string, string>>,
    private readonly fallbackLocale = "en"
  ) {}

  t(key: string, params: TranslationParams = {}, locale = this.fallbackLocale): string {
    const template = this.catalogs[locale]?.[key] ??
      ↪ this.catalogs[this.fallbackLocale]?.[key] ?? key;
    return Object.entries(params).reduce(
      (message, [paramKey, value]) => message.replaceAll(`${paramKey}`, String(value)),
      template
    );
  }
}
```

Example Kanbien usage

```
const subject = translator.t(
  "notifications.welcome.subject",
  { name: user.name },
  user.locale
);
```

Enterprise concerns

- translation workflow
- pluralization
- right-to-left support
- localized notifications
- regulatory language requirements

Architectural warning

Keep the core module focused on stable contracts and cross-cutting primitives. Put concrete provider integrations in `platform/`. Put product-specific behaviour in `apps/` or feature packages.

localization/ deep dive

Why it exists

Users and enterprises expect dates, money, numbers, and timezones to be correct for their region.

How it works internally

- Store canonical timestamps in UTC.
- Resolve user or tenant locale and timezone.
- Format values at presentation or notification boundaries.
- Use currency codes and minor units carefully.

Example implementation

```
export interface LocalizationService {
  formatDate(date: Date, locale: string, timeZone: string): string;
  formatDateTime(date: Date, locale: string, timeZone: string): string;
  formatCurrency(amountMinor: number, currency: string, locale: string): string;
}

export class IntlLocalizationService implements LocalizationService {
  formatDate(date: Date, locale: string, timeZone: string): string {
    return new Intl.DateTimeFormat(locale, { dateStyle: "medium", timeZone }).format(date);
  }

  formatDateTime(date: Date, locale: string, timeZone: string): string {
    return new Intl.DateTimeFormat(locale, { dateStyle: "medium", timeStyle: "short",
      ↪ timeZone }).format(date);
  }

  formatCurrency(amountMinor: number, currency: string, locale: string): string {
    return new Intl.NumberFormat(locale, { style: "currency", currency
      ↪ }).format(amountMinor / 100);
  }
}
```

Example Kanbien usage

```
const generatedAt = localization.formatDateTime(
  report.generatedAt,
  user.locale ?? "en-GB",
  user.timeZone ?? "Europe/Dublin"
);
```

Enterprise concerns

- regional compliance
- timezone correctness
- currency precision
- tax/invoice localization

- multi-region operations

Architectural warning

Keep the core module focused on stable contracts and cross-cutting primitives. Put concrete provider integrations in `platform/`. Put product-specific behaviour in `apps/` or feature packages.

Part 5: Events, Messaging, and Async Systems

This area causes confusion because the terms are related but not interchangeable.

Event-driven architecture

Event-driven architecture means parts of the system communicate by publishing and reacting to events. Instead of a user service directly calling email, audit, analytics, CRM sync, and billing services, it publishes a fact such as `UserCreated`. Other components react independently.

The station-announcement analogy works well: “The train has arrived.” Passengers, cleaners, conductors, and station staff each react in their own way. The announcement does not command each person individually.

Domain events

A domain event is a business-meaningful fact that has already happened.

Good event names:

```
UserCreated
InvoicePaid
FileUploaded
ReportGenerated
TenantProvisioned
```

Poor event names:

```
CreateUser
SendEmail
UpdateInvoice
```

Those are commands, not events. Events are usually past tense.

Event buses

An event bus receives events and distributes them to subscribers. In core, `EventBus` should be an interface. Platform can implement it with an in-memory bus, Kafka, SNS/SQS, RabbitMQ, NATS, or an outbox-backed publisher.

Message queues

A message queue stores work until a worker is ready to process it. Queues are useful when work is slow, unreliable, bursty, retryable, or should be handled outside the request path.

Dead-letter queues

A dead-letter queue stores messages that failed repeatedly. It is the lost-and-found office for undeliverable work. Enterprise systems need DLQs because otherwise failures either disappear or retry forever.

Background workers

A background worker is a process that consumes jobs or messages and performs work outside the user request. Examples include email workers, report workers, file processing workers, and integration sync workers.

Async jobs

An async job is a named unit of work to be performed later. It usually travels through a queue and is executed by a worker.

Scheduled jobs

A scheduled job runs at a specific time or interval. Examples: nightly cleanup, weekly report generation, hourly billing sync.

How they relate

```
Domain event:
  "UserCreated happened."

Event bus:
  Distributes that fact.

Queue:
  Holds work resulting from that fact.

Async job:
  "Send welcome email to this user."

Worker:
  Executes the job.

DLQ:
  Stores failed jobs after retries.

Scheduled job:
  Creates work based on time instead of a user action.
```

What belongs in core, apps, platform, and infra

Belongs in packages/core:

```
DomainEvent type
EventEnvelope type
EventBus interface
Queue interface
Job interface
JobHandler interface
Retry policy types
DLQ metadata types
Scheduler interface
Correlation and causation IDs
Serialization and versioning conventions
```

Belongs in apps:

```
UserCreated event definition
Business decision to send a welcome email
Business decision to record analytics
Specific job handlers
Specific scheduled job definitions
```

Belongs in platform:

Kafka/SQS/RabbitMQ implementation
Worker runtime
Queue consumers
Scheduler runtime
Outbox publisher
Retry processor
DLQ tooling

Belongs in infra:

Actual topics
Actual queues
Actual DLQs
IAM permissions
Worker deployments
Cron resources
Autoscaling rules

Complete worked example: UserCreated

Target flow:

API
→ Event
→ Queue
→ Worker
→ Notification
→ Audit
→ Analytics

1. API receives request

```
export async function createUserRoute(request: RequestContext) {
  const principal = await request.auth.requirePrincipal();
  const tenant = request.tenant.require();

  await requireAllowed(request.authorizer, {
    principalId: principal.subject,
    tenantId: tenant.tenantId,
    action: "user.create",
    resource: { type: "user" },
  });

  const user = await createUserService.create({
    tenantId: tenant.tenantId,
    email: request.body.email,
    name: request.body.name,
    actorId: principal.subject,
    correlationId: request.correlationId,
  });

  return { status: 201, body: user };
}
```

2. App creates user and records an outbox event

```
export async function createUser(input: {
  tenantId: string;
  email: string;
  name: string;
  actorId: string;
  correlationId: string;
}) {
  return transactionManager.run(async (tx) => {
    const user = await userRepository.create({
      tenantId: input.tenantId,
      email: input.email,
      name: input.name,
    }, tx);

    await outbox.add({
      id: crypto.randomUUID(),
      event: {
        type: "UserCreated",
        version: 1,
        payload: { userId: user.id, email: user.email, name: user.name },
      },
      tenantId: input.tenantId,
      actorId: input.actorId,
      occurredAt: new Date(),
      correlationId: input.correlationId,
    }, tx);

    return user;
  });
}
```

The outbox prevents this failure mode:

Save user to database
Publish event
Event publish fails
Database says user exists, but the rest of the system never knows

3. Event publisher publishes UserCreated

```
const events = await outbox.nextBatch();

for (const envelope of events) {
  await eventBus.publish(envelope);
  await outbox.markPublished(envelope.id);
}
```

4. Event subscriber creates a queue job

```
export async function onUserCreated(envelope: EventEnvelope<{
  userId: string;
  email: string;
  name: string;
}>) {
  await jobs.enqueue(
```



```

    "notifications.sendWelcomeEmail",
    {
      userId: envelope.event.payload.userId,
      email: envelope.event.payload.email,
      name: envelope.event.payload.name,
    },
    {
      tenantId: envelope.tenantId,
      idempotencyKey: `welcome-email:${envelope.event.payload.userId}`,
    }
  );
}
}

```

5. Worker processes the job

```

export const sendWelcomeEmailHandler: JobHandler<{
  userId: string;
  email: string;
  name: string;
}> = {
  type: "notifications.sendWelcomeEmail",

  async handle(job, context) {
    await notifications.send({
      type: "user.welcome",
      channel: "email",
      recipient: { userId: job.payload.userId, email: job.payload.email },
      template: "welcome_user",
      data: { name: job.payload.name },
      tenantId: job.tenantId,
    });
  },
};

```

6. Audit is recorded

```

await audit.record({
  action: "user.created",
  actor: { id: input.actorId, type: "user" },
  tenantId: input.tenantId,
  resource: { type: "user", id: user.id },
  occurredAt: new Date(),
  correlationId: input.correlationId,
});

```

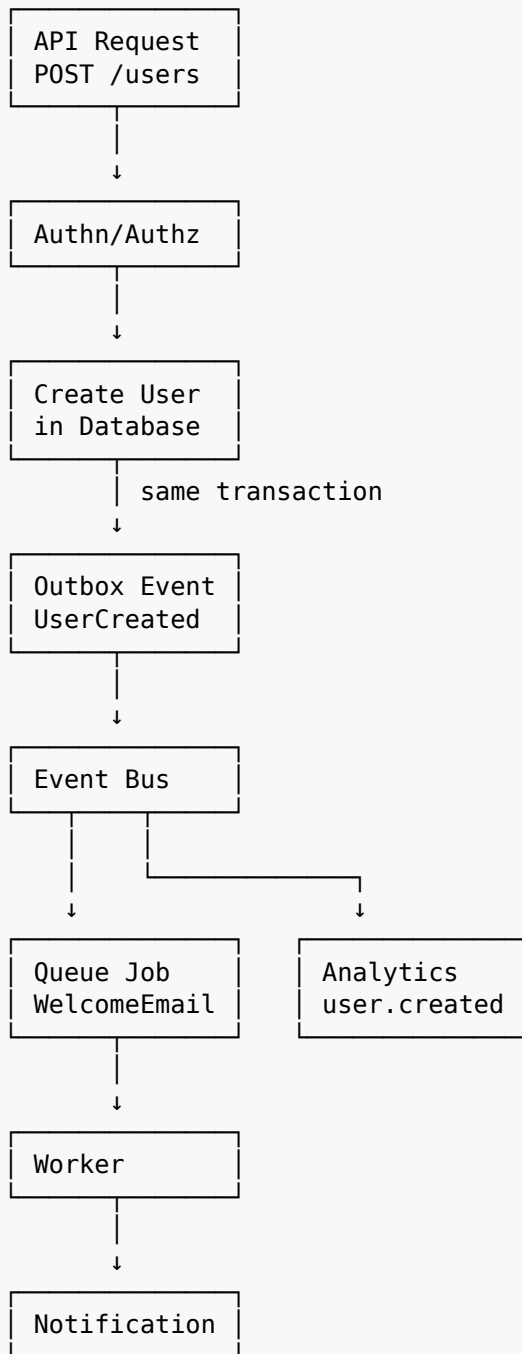
7. Analytics is tracked

```

await analytics.track({
  name: "user.created",
  userId: user.id,
  tenantId: input.tenantId,
  properties: { source: "admin_create" },
  occurredAt: new Date(),
});

```

Full flow diagram



Audit is recorded during the state change and can also be enriched by event consumers.

Part 6: Reporting vs Analytics vs Audit

These terms are often confused because each involves data about what happened. The distinction is purpose, audience, data source, ownership, and retention.

Reporting

Reporting produces structured, repeatable operational or business outputs. The audience is usually customers, admins, operations, finance, or account teams. Data usually comes from application databases, read models, or a warehouse. Retention depends on the report and customer contract. Enterprise concerns include access control, export audit, correctness, freshness, and large data volumes.

Examples:

```
Monthly tenant usage report
CSV export of invoices
Compliance report
User activity report
```

Analytics

Analytics explains product and user behaviour. The audience is product, growth, leadership, and data teams. Data comes from product instrumentation, event tracking, and sometimes warehouse models. Enterprise concerns include consent, privacy, event taxonomy, PII minimization, and regional data rules.

Examples:

```
Signup conversion
Feature adoption
Activation funnel
Retention by tenant segment
```

Business Intelligence

Business Intelligence combines data across business domains to support strategic decisions. The audience is executives, finance, operations, customer success, and data teams. Data usually comes from a data warehouse fed by product, billing, CRM, support, marketing, and finance systems. Enterprise concerns include metric definitions, lineage, governance, and data quality.

Examples:

```
Revenue by region
Churn by segment
Expansion revenue by cohort
Support load by customer tier
```

Audit

Audit is a formal accountability record. It answers who did what, when, to what resource, in which tenant, and under whose authority. The audience is security, compliance, customers, auditors, and sometimes legal teams. Audit requires stronger integrity and retention controls than analytics or logs.

Examples:

```
Admin changed role
User exported data
```

File downloaded
Support agent impersonated customer
Permission denied

Observability

Observability is the ability to understand system behaviour from emitted signals. It combines logs, metrics, traces, and sometimes events. The audience is engineering and SRE. It is used for debugging, incident response, and system understanding.

Monitoring

Monitoring detects health problems and alerts humans or automation. It focuses on metrics, health checks, probes, SLOs, and alerting.

Examples:

API error rate too high
Queue depth growing
Database latency spiking
Worker failure rate increasing

Logging

Logging records operational details from application execution. Logs help developers and operators debug specific paths. They are not a substitute for audit, analytics, or monitoring.

Relationship diagram

```
Business questions
├─ Reporting
├─ Analytics
└─ Business Intelligence

Accountability
└─ Audit

System health and debugging
├─ Monitoring
├─ Logging
└─ Observability
```

Quick comparison

Reporting:
Purpose: structured outputs
Audience: customers, admins, operations
Source: app DB, read models, warehouse
Owner: product/operations
Retention: medium to long

Analytics:
Purpose: behaviour insight
Audience: product, growth, leadership
Source: tracking events

Owner: product/data
Retention: privacy-dependent

Business Intelligence:

Purpose: cross-business analysis
Audience: executives, finance, data teams
Source: warehouse and marts
Owner: data/finance/ops
Retention: long

Audit:

Purpose: accountability evidence
Audience: security, compliance, customers, auditors
Source: important app/security actions
Owner: security/compliance/platform
Retention: long and policy-driven

Observability:

Purpose: understand system behaviour
Audience: engineering/SRE
Source: logs, metrics, traces
Owner: engineering/platform
Retention: usually shorter than audit

Monitoring:

Purpose: detect health problems
Audience: engineering/SRE/operations
Source: metrics and health checks
Owner: engineering/platform
Retention: short to medium

Logging:

Purpose: execution detail
Audience: developers/SRE
Source: app/runtime logs
Owner: engineering
Retention: short to medium

Part 7: Multi-Tenancy

What multi-tenancy is

Multi-tenancy means one software platform serves multiple customer organisations while keeping their data, actions, configuration, and permissions isolated. Each customer organisation is a tenant.

A hotel is the right analogy. Guests share reception, lifts, utilities, and building services, but each room must remain private.

Tenant isolation models

Shared database, shared schema

Every table has a tenant_id column.

```
SELECT * FROM users WHERE tenant_id = 'tenant_123';
```

Pros:

- Cheapest
- Simplest early operations
- Easy deployment
- Works for many small tenants

Cons:

- Tenant leakage risk if queries are wrong
- Harder noisy-neighbor isolation
- Harder customer-specific backup/restore
- Harder data residency separation

This is often the best v1 model if implemented carefully.

Shared database, separate schemas

Each tenant has a separate database schema.

```
tenant_acme.users  
tenant_globex.users
```

Pros:

- Better logical isolation
- Some customer-specific restore possibilities
- Some customization possible

Cons:

- More migration complexity
- More operational complexity
- Can become painful with many tenants

Separate database per tenant

Each tenant receives its own database.

Pros:

- Stronger isolation
- Better enterprise story

Easier customer-specific backup/restore
Better noisy-neighbor isolation
Possible customer-specific region

Cons:

More expensive
More operationally complex
Harder migrations
Harder cross-tenant analytics

Separate deployment per tenant

Each tenant receives its own app deployment, database, queues, storage, region, and operational boundary.

Pros:

Maximum isolation
Strong regulated-industry posture
Customer-specific release control

Cons:

Most expensive
Hardest to operate
Version drift risk
Slowest operational scale

Hybrid model

Many mature SaaS platforms use a hybrid model:

Small tenants: shared database and shared schema
Large tenants: dedicated database
Highly regulated tenants: dedicated region or deployment

Kanbien should not build all of this on day one, but it should avoid designs that block this future.

Security implications

Tenant isolation must prevent:

Tenant A seeing Tenant B's data
Tenant A triggering jobs for Tenant B
Tenant A accessing Tenant B's files
Tenant A receiving Tenant B's notifications
Tenant A's admins managing Tenant B users

Tenant context should flow through authn, authz, persistence, events, queues, files, audit, analytics, logging, monitoring, and notifications.

Performance implications

Shared tenants can create noisy-neighbor problems. For example, Tenant A imports 5 million records and Tenant B's dashboard slows down.

Mitigations include:

```
tenant-level rate limits
queue partitioning
database indexes on tenant_id
read replicas
per-tenant quotas
background job fairness
dedicated infrastructure for large tenants
```

Where tenancy support belongs

Core should define `TenantId`, `TenantContext`, tenant resolution interfaces, tenant propagation helpers, and isolation errors.

Apps should own tenant onboarding, tenant settings, tenant billing logic, and tenant-specific product behaviour.

Platform should implement tenant resolution, database routing, row-level security, tenant-specific storage routing, and region selection.

Infrastructure should provision databases, schemas, buckets, queues, regions, IAM boundaries, and networks.

Part 8: Internationalization and Localization

i18n

Internationalization means designing software so it can support multiple languages and regions. It is preparation.

Examples:

```
Use translation keys instead of hardcoded strings
Avoid string concatenation
Support pluralization
Support locale negotiation
```

l10n

Localization means adapting the software for a specific language, country, or region. It is execution.

Examples:

```
English UK copy
French copy
German date formatting
Euro currency
Arabic right-to-left layout
Japanese address formatting
```

Translations

Translations are the actual text mappings.

```
{
  "notifications.welcome.subject": "Welcome to Kanbien, {name}"
}
```

Core should define the translation mechanism. Apps should own most product copy. The design system should own UI primitive copy and rendering behaviour.

Locale management

Locale is the user or tenant's language and region preference.

Examples:

```
en
en-GB
en-US
fr-FR
de-DE
ja-JP
```

Locale can come from user profile, tenant settings, browser headers, request parameters, or a system default.

Currency handling

Currency handling is not just display formatting. It involves currency code, minor units, rounding, invoices, tax, and sometimes exchange rates.

A v1 system can start with formatting, but financial correctness eventually requires a proper money model.

Timezone handling

Store canonical timestamps in UTC. Render them in the user or tenant timezone.

Stored:

2026-06-22T09:00:00Z

Displayed to Dublin user:

22 Jun 2026, 10:00

Displayed to New York user:

22 Jun 2026, 05:00

Do not store local timestamps as the source of truth.

Regional formatting

Regional formatting includes dates, times, numbers, currencies, percentages, phone numbers, addresses, units, and week-start rules.

Responsibility split

packages/core:

- Locale type
- Translator interface
- LocalizationService
- Date/time and currency formatting primitives
- Validation message localization mechanism

packages/design-system:

- LocalizedText
- DateDisplay
- CurrencyDisplay
- RTL layout support
- Localized form error rendering

apps:

- Translation catalogs
- Product copy
- User locale preferences
- Tenant locale defaults
- Notification templates

Part 9: Building Kanbien v1

Because Kanbien is being built by one person, the goal is not to create a large enterprise platform immediately. The goal is to create clean seams early and fully implement only what is necessary.

Create clear seams early.

Fully implement only what is needed now.

Avoid pretending Kanbien is already a 500-person platform company.

Must Have on day one

config

Use typed, validated configuration from the start. Avoid scattered `process.env` access. Missing config should fail fast at startup.

logging

Use structured logs with correlation IDs and redaction basics. Debugging production without structured logs is slow and expensive.

validation

Standardize validation result and API error shapes. This prevents every endpoint from inventing a different error model.

authn

Define a consistent `Principal` model and authentication interface, even if the first implementation is simple.

authz

Define authorization decisions early. Start with simple role-based checks, not a complex policy engine.

tenancy, lightweight

Even if Kanbien starts single-tenant, introduce `TenantId` and `TenantContext` early. Retrofitting tenancy after data exists is painful.

persistence

Define transactions, pagination, repository conventions, and outbox support if using events.

security

Add redaction, secure token generation contracts, secret hashing contracts, and sensitive-field conventions.

audit

Record sensitive actions from the beginning: role changes, security events, data exports, deletes, and privileged actions.

events, minimal

Define EventEnvelope, versioned events, correlation IDs, and an in-process or outbox-backed bus. Do not start with Kafka unless the need is real.

Should Have as the platform grows

Add queues and async jobs when email, file processing, report generation, or integrations become common.

Add notifications when multiple message types, localization, preferences, or delivery tracking matter.

Add files when users upload files, reports are generated, or exports are stored.

Add monitoring when production users depend on the system, especially once workers and external dependencies exist.

Add analytics when product adoption, activation, retention, and usage signals matter.

Add reporting when reports become more than simple UI queries or exports need async generation and access control.

Add i18n and localization before international usage becomes urgent. At minimum, avoid assuming one timezone, one currency, one date format, and one language forever.

Enterprise Future

When Kanbien has multiple customers, teams, regions, or products, invest in:

```
centralized tenant isolation enforcement
customer-specific roles and permissions
SSO/SAML/OIDC
SCIM provisioning
advanced audit search and export
tamper-resistant audit storage
real event bus
durable queues and DLQs
worker autoscaling
outbox/inbox patterns
schema registry for events
data warehouse
BI semantic layer
feature flag governance
regional data residency
customer-managed encryption keys
localization workflow
compliance evidence automation
operational runbooks
```

Do not build the enterprise future on day one. But do create boundaries that let Kanbien reach it without a rewrite.

Part 10: Final Architecture Review

Ideal packages/core folder structure

```
packages/core/  
  package.json  
  tsconfig.json  
  README.md  
  
  src/  
    index.ts  
  
    shared/  
      ids.ts  
      result.ts  
      errors.ts  
      clock.ts  
      context.ts  
      lifecycle.ts  
  
    config/  
      index.ts  
      types.ts  
      provider.ts  
      errors.ts  
  
    logging/  
      index.ts  
      types.ts  
      logger.ts  
      redaction.ts  
      testing.ts  
  
    monitoring/  
      index.ts  
      types.ts  
      metrics.ts  
      health.ts  
      testing.ts  
  
    security/  
      index.ts  
      redaction.ts  
      secrets.ts  
      crypto.ts  
      rate-limit.ts  
      errors.ts  
  
    authn/  
      index.ts  
      types.ts  
      authenticator.ts  
      principal.ts  
      errors.ts  
  
    authz/  
      index.ts  
      types.ts
```

```
authorizer.ts
permissions.ts
policy.ts
errors.ts

tenancy/
  index.ts
  types.ts
  tenant-context.ts
  tenant-resolver.ts
  errors.ts

validation/
  index.ts
  types.ts
  validators.ts
  schemas.ts
  errors.ts

persistence/
  index.ts
  types.ts
  transaction.ts
  pagination.ts
  repository.ts
  outbox.ts
  errors.ts

events/
  index.ts
  types.ts
  envelope.ts
  event-bus.ts
  versioning.ts
  testing.ts
  errors.ts

queues/
  index.ts
  types.ts
  queue.ts
  retry.ts
  dead-letter.ts
  testing.ts
  errors.ts

async-jobs/
  index.ts
  types.ts
  scheduler.ts
  handler.ts
  status.ts
  retry.ts
  errors.ts

notifications/
  index.ts
  types.ts
```

```
notification-service.ts
templates.ts
preferences.ts
errors.ts

files/
  index.ts
  types.ts
  file-storage.ts
  content-types.ts
  access.ts
  errors.ts

audit/
  index.ts
  types.ts
  audit-service.ts
  actor.ts
  resource.ts
  changes.ts
  errors.ts

reporting/
  index.ts
  types.ts
  report-generator.ts
  formats.ts
  exports.ts
  errors.ts

analytics/
  index.ts
  types.ts
  analytics.ts
  taxonomy.ts
  consent.ts
  errors.ts

i18n/
  index.ts
  types.ts
  translator.ts
  message-catalog.ts
  fallback.ts
  errors.ts

localization/
  index.ts
  types.ts
  date-time.ts
  currency.ts
  locale.ts
  timezone.ts
  errors.ts

testing/
  in-memory-event-bus.ts
  in-memory-queue.ts
```

```
noop-logger.ts
fake-clock.ts
fake-authenticator.ts
fake-authorizer.ts
```

Why each folder exists and what consumes it

```
shared/
  Common primitives: IDs, results, errors, clocks, context.
  Consumed by apps, platform, tests.

config/
  Configuration access and validation.
  Consumed by all runtime apps and platform services.

logging/
  Structured logs, redaction, logger contracts.
  Consumed by apps, workers, platform.

monitoring/
  Health checks and metrics contracts.
  Consumed by apps, workers, platform.

security/
  Defensive primitives and security contracts.
  Consumed by apps and platform security adapters.

authn/
  Principal and authentication contracts.
  Consumed by apps and platform identity integrations.

authz/
  Permission and policy decision contracts.
  Consumed by apps and feature packages.

tenancy/
  Tenant context and isolation primitives.
  Consumed by nearly everything.

validation/
  Standard validation and error models.
  Consumed by APIs, apps, and possibly frontend packages.

persistence/
  Transactions, pagination, repositories, outbox contracts.
  Consumed by apps and platform persistence implementations.

events/
  Event envelopes, buses, versioning.
  Consumed by apps, workers, platform messaging.

queues/
  Queue messages, consumers, retry, DLQ concepts.
  Consumed by workers and platform queue adapters.

async-jobs/
  Background job contracts and lifecycle.
```



```

Consumed by apps and workers.

notifications/
  Notification contracts and delivery model.
  Consumed by apps, workers, and provider adapters.

files/
  File storage and metadata contracts.
  Consumed by apps and platform storage adapters.

audit/
  Audit event model and recorder interface.
  Consumed by apps and platform audit storage.

reporting/
  Report request, generation, and export contracts.
  Consumed by apps and workers.

analytics/
  Product tracking and identity contracts.
  Consumed by apps and analytics adapters.

i18n/
  Translation and message catalog contracts.
  Consumed by apps, notifications, and design system.

localization/
  Date, time, locale, currency, and regional formatting.
  Consumed by apps, notifications, and design system.

testing/
  In-memory and no-op implementations.
  Consumed by apps, harness, and CI tests.

```

How core connects to apps, platform, harness, and infrastructure

Apps import core types and interfaces and compose them into product workflows.

```

import {
  Authorizer,
  AuditService,
  EventBus,
  TenantContext,
  Logger,
} from "@kanbien/core";

```

Platform implements core contracts.

```

packages/core/files
  FileStorage interface

platform/storage-s3
  S3FileStorage implements FileStorage

```

Harness uses fake or in-memory implementations to run Kanbien locally without every cloud dependency.

Infrastructure provisions resources but should not be imported by core.

```
core:
  "A queue must support send and consume."

platform:
  "This implementation uses SQS."

infra:
  "Here is the actual SQS queue in eu-west-1."
```

Most common mistakes founders make

Founders often overbuild platforms before the product is proven. They build generic workflow engines, multi-region abstractions, full plugin systems, complex policy languages, and enterprise reporting frameworks too early.

The opposite mistake is underinvesting in security, audit, and tenancy. For enterprise SaaS, those are not decorations. Even v1 should preserve actor ID, tenant ID, authorization checks, audit records, redacted logs, and consistent validation.

Another founder mistake is confusing reuse with architecture. Some duplication is cheaper than bad abstraction.

Most common mistakes engineers make

Engineers often create abstract frameworks too early. A good abstraction reduces real complexity now and preserves flexibility later. A bad abstraction turns imagined future needs into present complexity.

Other common mistakes:

```
leaking infrastructure into core
ignoring dependency boundaries because it is faster
treating logs, audit, analytics, and monitoring as the same thing
using global singletons everywhere
failing to version events and contracts
creating a shared package without ownership
```

Principles to guide Kanbien for the next 10 years

Core defines contracts. Platform provides implementations. Infrastructure provides resources. Apps decide product workflows.

Keep dependency direction clean:

```
Apps can depend on core.
Platform can depend on core.
Core must not depend on apps, platform, or infra.
```

Prefer boring, explicit architecture over trendy architecture.

Design for understandability, testability, replaceability, operational clarity, security, and maintainability.

Start simple, but leave seams. Use simple roles, simple events, simple audit, simple queues, and simple file storage first. The seams let you replace implementations later.

Treat tenancy as a first-class concern. Retrofitting tenancy later is painful.

Treat audit as evidence, not logging.

Do not put product workflows in core. Core provides ingredients. Apps cook the meal.

Make important concepts visible in types: `TenantId`, `UserId`, `Principal`, `Permission`, `AuditEvent`, `EventEnvelope`, `Locale`, `Currency`, and `CorrelationId`.

Design for operations. A system is not finished when it compiles. It is finished when you can run it, observe it, debug it, secure it, recover it, audit it, explain it, and evolve it.

Good architecture is not the most elaborate architecture. For Kanbien, the best architecture is the one that keeps the core small, stable, well-owned, and boring enough that future teams can trust it.